A Controller Placement Algorithm Based on Density Clustering in SDN

Hou Xiaolan, Wu Muqing, Xu Weiyao
Beijing Laboratory of Advanced Information Networks
Beijing University of Posts and Telecommunications
Beijing, China
Email:{houxiaolan,wumuqing,bingbing2016}@bupt.edu.cn

***Abstract*—Software-defined networks(SDNs) are improving the controllability and flexibility of networks as an innovative network architecture paradigm. The reasonable placement of the controller in SDN would effectively improve the performance of entire network. In this paper, we study clustering-based controller placement algorithm. The clustering by fast search and find of density peaks (CFSFDP) algorithm is a density-based clustering algorithm with less initial parameters and faster execution speed. However, the CFSFDP algorithm does not consider other performance indicators in the network and the number of the clustering centers needs to be manually determined. In this paper, we propose a controller placement algorithm in SDN to reduce propagation delay and communication consumption in the intra-controller and inter-controller. And the number and the location of controllers are discovered automatically through the inflection point of decision graph. We evaluate the performance of the proposed algorithm on internet2 OS3E network topology. The experimental results show that the proposed algorithm provides better performance than the traditional approaches in terms of communication consumption and propagation delay.**

***Index Terms*—SDN, controller placement, density-based clustering, communication consumption, propagation delay**

I. Introduction

Software-defined network(SDN) [1] is becoming an innovative paradigm for next-generation networks. The defining feature of SDN compared to traditional networks is that it decouples the network control plane from the data plane, simplifies network management, and makes it more flexible. With the gradual increase in SDN deployment in large-scale network topology, the deployment of single-controller is difficult to meet the actual deployment requirements. The deployment of distributed multi-controller is an effective way to solve the scalability of control plane.

In the scenario of distributed multi-controller deployment, the strategy of controller placement faces severe challenges and ordeals [2]. At present, there have been some architectures for solving the scalability of SDN control plane, such as HyperFlow [3], Onix [4] , etc. HyperFlow adopts the deployment of multiple controllers to control switches. Each controller obtains global network view and keeps synchronization with other controllers. Each controller only needs to control the switches in its own domain. Onix also proposes a deployment scheme of distributed SDN controller, which realizes the distributed deployment of SDN in large-scale network and improves the reliability, extensibility and universality of control plane. The network architecture of Onix consists of four parts: physical infrastructure layer at the bottom, the network connection layer, Onix layer and the network control layer. In the process of solving the controller placement problem, the search algorithm determines the number and the location of controllers based on the performance requirements. But the solution of the controller placement problem is NP hard. Some search algorithms for controller placement problem [5]–[9] are proposed, such as random search algorithm, greedy search algorithm, game algorithm, integer programming algorithm, heuristic search algorithm, clustering algorithm and so on. The authors in [5] aim at the QoS-Guaranteed controller placement problem and propose three heuristic algorithms: network-partition-based algorithm, incremental greedy algorithm and primal-dual-based algorithm. The authors in [6] introduce a binary integer program (BIP) model to minimize energy consumption under the constraints of the delay and the load. Furthermore, a genetic heuristic algorithm is proposed to search a valid sub-optimal solution in large networks. The authors in [7] mainly focus on two aspects. One is the delay, communication overhead and load balancing between the controllers. The other is the delay and communication overhead between switches and controllers. The authors use bargaining game to find the balance between these objectives. The authors in [8] propose the clustering algorithm and the greedy algorithm to optimize controller placement problem for the network average reliability. The authors in [9] propose an approach named as Density Based Controller Placement (DBCP). The authors use a density-based switch clustering algorithm to divide the network into multiple sub-networks. The optimal number of controllers is automatically acquired by clustering based on density. And the authors in [10] propose the clustering by fast search and find of density peaks(CFSFDP) algorithm which provides a new idea for the design of clustering algorithm with less initial parameters and faster execution speed. However, the CFSFDP algorithm does not consider other performance indicators in the network and

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:14:41 UTC from IEEE Xplore. Restrictions apply.

the number of controllers needs to be manually determined. In the large scale networks, propagation delay and communication consumption of nodes are very important. In this paper, we consider propagation delay and communication consumption as the optimized targets based on the CFSFDP algorithm. Moreover, the number and location of controllers are obtained by calculating the change in the angle of the curve to find the inflection point of the curve.

The structure of the paper is as follows. In Section II, we discuss related work. In Section III, we introduce our proposed algorithm. In Section IV, experimental results are presented and discussed. In Section V, we complete this paper with a conclusion.

II. Related Work

Clustering analysis is a statistical analysis method for studying classification problem and has been successfully applied in different fields. The clustering algorithms are classified as density-based, grid-based, model-based, partitioning and hierarchical. The k-means [11] is an outstanding classical clustering algorithm. In k-means, data are partitioned into k clusters and these clusters are iteratively optimized. The k-means can only find spherical clusters and is insensitive to noise. The precision of k-means is influenced by the number of clusters and the position of the initial centroids. The CFSFDP algorithm provides a new idea for the design of clustering algorithm with less initial parameters and faster execution speed. Compared with other density clustering algorithms, the CFSFDP algorithm has a novel idea and does not require iterative solution. The core idea of the algorithm is to characterize the clustering center. The authors believe that the clustering center has the following two characteristics. First, the density of the clustering center is large. And the clustering center is surrounded by neighbors with lower local density. Second, the distance between the data points with other denser is greater. The DBCP algorithm proposes a density-based clustering algorithm to solve controller placement problem. The size of each sub-network can be determined by the capacity of the controller deployed. However, the distance between two switches is the shortest path hops in this algorithm without considering other parameters such as delay and communication consumption. And the optimal number of controllers is only calculated by the mean value of σ, without considering ρ and theoretical proof.

In this paper, we study controller placement problem based on the CFSFDP algorithm. The CFSFDP algorithm calculates related parameters based on the distance between nodes. And the DBCP algorithm defines the number of the shortest path hops as the distance between nodes. The two algorithms do not consider other network performance. In this paper, we consider propagation delay and communication consumption as the optimized targets. When choosing the clustering center, we define the propagation delay between the nodes as the distance between the nodes. The clustering centers are surrounded by the nodes with small delay. When the non-clustering center nodes are categorized, the node belongs to the clustering center with the least communication consumption among the clustering centers with larger consumption density. In this way, the delay and communication consumption between clustering centers and other nodes are optimized. In addition, it is difficult to determine the number of controllers when the network size and topology are not clear, and the accurate number of controllers can greatly improve the network performance. In this paper, we obtain the number of controllers by calculating the change in the angle of the γ curve.

III. Algorithm Formulation

In this paper, we research the controller placement problem with propagation delay and communication consumption based on the CFSFDP algorithm. The clustering center of the CFSFDP algorithm has two characteristics: its own density is large, and the relative distance between other denser data points is larger. However, we define propagation delay between the nodes as the distance between the nodes. Then the selected clustering center indicates that it is surrounded by a large number of nodes with little delay, and the delay between the clustering centers is large. When the clustering center has been selected, the non-clustering center node belongs to the clustering center with the smallest communication consumption among the clustering centers with larger consumption density. We use the weighted graph $\mathbf{G}\left(\mathbf{V},\mathbf{E}\right)$ to present the network topology of SDN, where $\mathbf{V}$ is represented by the set of nodes, and $\mathbf{E}$ is represented by the set of edges. In the process of selecting the clustering center, we represent the delay matrix between nodes as the distance matrix between the nodes.

A. The Parameters of Distance and Density

a) Local Density: Local density includes two calculation methods: cut-off kernel and Gaussian kernel.

- Cut-off kernel

$$\rho_i = \sum_j \chi(d_{ij} - d_c) \tag{1}$$

where $\chi\left(x\right) = \begin{cases} 1, x < 0; \\ 0, x \geq 0, \end{cases}$, parameters $d_c > 0$ is the cut-off distance. ρ_i represents the number of points that are less than d_c to point i.

- Gaussian kernel

$$\rho_i = \sum_{j \in V/\{\mathrm{i}\}} e^{-\left(\frac{d_{i,j}}{d_c}\right)^2} \tag{2}$$

From (1) and (2), we can see that cut-off kernel is a discrete value and Gaussian kernel is a continuous value. In this paper, we calculate the local density by (2), so that the probability of the different density points with the same local density is very small.

b) The Distance δ: Let $\{q_i\}_{i=1}^{|\mathrm{V}|}$ represent a subscript of a descending order of $\{\rho_i\}_{i=1}^{|\mathrm{V}|}$. That is, it satisfies:

$$\rho_{q_1} \geq \rho_{q_2} \geq ... \geq \rho_{q_{|\mathrm{V}|}} \tag{3}$$

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:14:41 UTC from IEEE Xplore. Restrictions apply.

The distance δ can be defined:

$$\delta_{q_i} = \begin{cases} \min\limits_{\substack{q_j \\ j<i}}\{d_{q_i q_j}\}, i \geq 2; \\ \max\limits_{j \geq 2}\{\delta_{q_j}\}, i = 1. \end{cases} \tag{4}$$

B. The Determination of the Number and Location of Controller

According to the definition of δ , when the node q_i has the maximum local density, δ_{q_i} represents the minimum distance between node q_i and the nodes of the local density greater than q_i node. The clustering centers are defined as these nodes that have large ρ and δ values at the same time. The authors in [10] determine the clustering center by the comprehensive consideration value γ. And the γ is defined as $\gamma_i = \rho_i \cdot \delta_i$. However, there are two problems with this definition. On the one hand, the values of ρ and δ may be in different orders of magnitude. To solve this problem, ρ and δ are normalized, so that they can be treated equally. On the other hand, the nodes at different densities may produce the same δ, which cause difficulty when the initial clustering center is selected. Through the analysis of the algorithm, the selected initial clustering center is more dependent on the δ. So it is easier to extract the clustering center in the low density area by adding the weight of the δ. So, after ρ and δ are normalized, the calculation equation for the decision value is modified as follows:

$$\gamma_i = \overline{\rho_i} \cdot \overline{\delta_i}^{3/4} \tag{5}$$

The larger the γ value, the more likely it is to be a clustering center. We sort the γ in descending order as shown in the Fig. 1. It can be seen that the γ value of the non-clustering center is relatively smooth, and the γ value has a significant jump when the non-clustering center transitions to the clustering center. The jump can be clearly seen. The key is how to automatically determine the jumping point without human intervention.

We reduce the abscissa in Fig. 1 by a factor of 100, as shown in Fig. 2. We can see that the curve of the γ value becomes the fold line, and the inflection point of the curve is at the vertex of the fold line. We only need to find the vertex to determine the inflection point of γ. We regard the descending order chart of the γ value after reducing the abscissa 100 times as the fold line A-B-C, as shown in Fig. 3. It is obvious that $\angle 1 > \angle 2 > \angle 3 < \angle 4 < \angle 5$. So, the cosine of $\angle 3$ at the inflection point B is the largest. Similarly, we can conclude that the cosine of the angle formed between the inflection point and the maximum and minimum value of γ is the largest. Let B' be any point on the γ curve except the maximum A and minimum C. We calculate the cosine value of $\angle \mathrm{AB'C}$ to find the inflection point B by discovery the maximum cosine value. The number of points with value greater than the inflection point γ value is considered to be the optimal number of controllers. The location of points with value greater than the inflection point γ value are where the controllers are deployed. The algorithm 1 shows the selection process of the clustering centers.

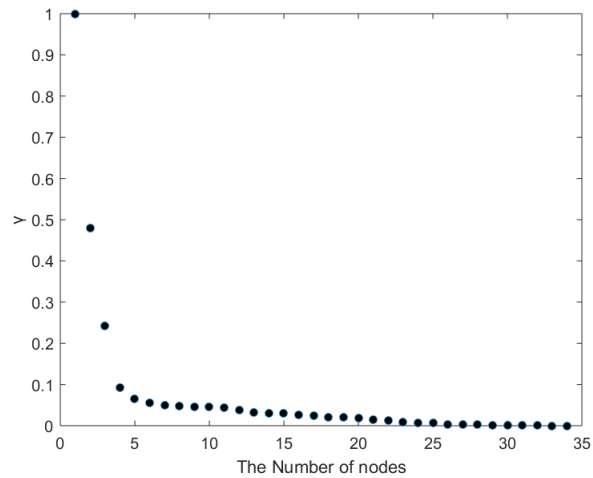

Fig. 1. The descending order of γ values.

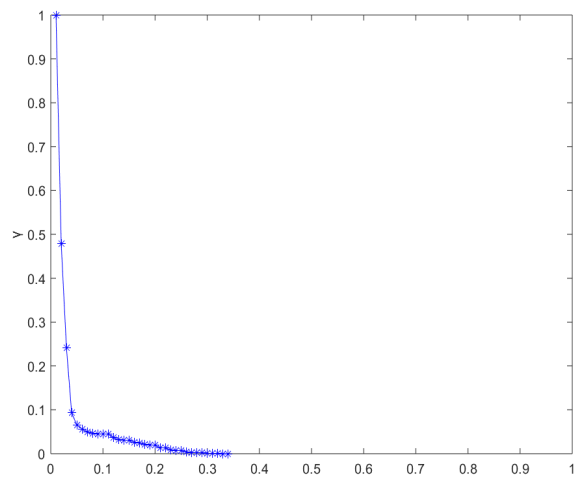

Fig. 2. The descending order of γ value after reducing the abscissa in fig.1 100 times.

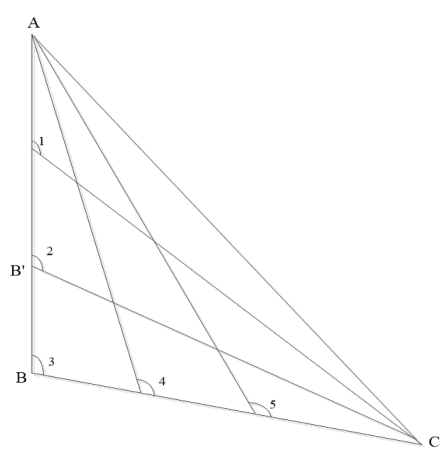

Fig. 3. The fold line A-B-C.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:14:41 UTC from IEEE Xplore. Restrictions apply.

TABLE I
THE PROCESS OF SELECTING CLUSTERING CENTERS

Algorithm 1 the process of selecting clustering centers
1. Input: $\mathbf{G}=(\mathbf{V},\mathbf{E})$, cutoff distance d_c , delay matrix D.
2. Calculate the delay density of each switch $\{\rho_i\}_{i=1}^{\lvert V\rvert}$ based on the delay distance matrix D.
3. Calculate the $\{\delta_i\}_{i=1}^{\lvert V\rvert}$ according to (4).
4.$\{\rho_i\}_{i=1}^{\lvert V\rvert}$ and $\{\delta_i\}_{i=1}^{\lvert V\rvert}$ are normalized separately to get $\{\overline{\rho_i}\}_{i=1}^{\lvert V\rvert}$ and $\left\{\overline{\delta_i}\right\}_{i=1}^{\lvert V\rvert}$.
5. $\gamma_i=\overline{\rho_i}\cdot\overline{\delta_i}^{3/4}$.
6. Sort γ in descending order after reducing the abscissa of γ 100 times.
7. Find the inflection point B by the maximum cosine.
8. The number of nodes with γ value greater than the inflection point γ value is considered to be the optimal number of controllers.
9. The location of nodes with γ value greater than the inflection point γ value is considered to be where the controller is deployed.
10. Output: the number of controllers K and the deployment position of the controller P.

TABLE II
THE PROCESS OF CLASSIFICATION OF NON-CLUSTERING CENTERS

Algorithm 2 the process of classification of non-clustering centers
1. Input: $\mathbf{G}=(\mathbf{V},\mathbf{E})$, the number of controller K, the deployment position of the controller P, the communication consumption matrix C, cutoff distance d'_c.
2. Calculate the communication consumption density of each switch $\{\rho'_i\}_{i=1}^{\lvert V\rvert}$ based on the communication consumption matrix C.
3. for $i\in V/P$.
4. $\mathbf{H}\in\phi$, $\mathbf{M}\in\phi$.
5. for $j\in P$
6. if $\rho'_i<\rho'_j$
7. $\mathbf{H}=\mathbf{H}\cup j$
8. $\mathbf{M}=\mathbf{M}\cup c_{i,j}$
9. end
10. end
11. if $\mathbf{H}=\phi$
12. i belongs to the controller corresponding to $Min\left(c_{i,P}\right)$.
13 else
14. i belongs to the controller corresponding to $Min(M)$.
15 end
16 end
17. Output: The set of controllers corresponding to the switches.

C. The classification of switches

Although SDN has the advantage of flexibility in the network, it brings the additional communication consumption to the control plane and the data plane when the flow arrives and SDN updates the global state information in real time. Therefore, when classifying non-clustering center nodes, we use communication consumption between controllers and switches as performance evaluation index. In the CSFSDP algorithm, when the nodes of the non-clustering center are clustered, the nodes are traversed in the order of ρ value from large to small. The algorithm is to expand each cluster layer by layer through $\{\mathrm{n}_i\}_{i=1}^{\lvert V\rvert}$. And $\{\mathrm{n}_i\}_{i=1}^{\lvert V\rvert}$ indicates the number of nodes closest to v_i in the nodes with larger local density than v_i. When the non-clustering center is classified, we represent the communication consumption matrix $\{c_{i,j}\}$ between nodes as a distance matrix $\{d_{i,j}\}$ and consider the consumption density and communication consumption between nodes and clustering centers. The non-clustering central nodes select the clustering center with the least consumption among the clustering centers with larger consumption density. We deploy the SDN controllers at the clustering centers. The controller manages all switches in the area it controls. The consumption between the non-clustering center nodes and the clustering center means the consumption between controllers and switches. The algorithm 2 shows the process of classification of non-clustering centers.

IV. PERFORMANCE EVALUATION

In this section, we compare the performance of our proposed algorithm with the random deployment algorithm, k-mean algorithm, CFSFDP algorithm and DBCP algorithm in terms of propagation delay and communication consumption. We use MATLAB to simulate and use Internet2 OS3E network with 34 nodes and 41 edges. Each node represents a switch and the connection between the nodes represents the link between the switches. We select the d_c, so that the average number of neighbors per node is 2% of the total number of nodes. We evaluate the performance from the average delay, the maximum delay, the average consumption, and the maximum consumption for the intra-controllers and the inter-controllers.

A. The Evaluation of Propagation Delay

1) Average Delay in the Intra-controller: For a sub-network graph, the average propagation delay between switches in the kth intra-controller is calculated as:

$$T_k^{avg}=\frac{1}{\lvert V_k\rvert\cdot(\lvert V_k\rvert-1)/2}\sum_{j>i}^{\lvert V_k\rvert}\sum_{i=1}^{\lvert V_k\rvert-1}d(i,j) \tag{6}$$

where $d(i,j)$ is the propagation delay between switch i and switch j in the intra-controller. $\lvert\mathbf{V}_k\rvert$ represents the number of switches in the kth controller domain.

For the entire network, the average propagation delay between switches in the intra-controller is calculated as:

$$T_K^{avg}=\frac{1}{\sum\limits_{k=1}^{K}\frac{1}{\lvert V_k\rvert\cdot(\lvert V_k\rvert-1)/2}}\sum_{k=1}^{K}\sum_{j>i}^{\lvert V_k\rvert}\sum_{i=1}^{\lvert V_k\rvert-1}d(i,j) \tag{7}$$

2) Maximum Delay in the Intra-controller: For a sub-network graph, the maximum propagation delay between switches in the kth intra-controller is calculated as:

$$T_k^{max}=\max_{i\in V_k,j\in V_k/i}d(i,j) \tag{8}$$

For the entire network, the maximum propagation delay between switches in the intra-controller is calculated as:

$$T_K^{max}=\max_{k\in\{1,2,\ldots,K\},i\in V_k,j\in V_k/i}d(i,j) \tag{9}$$

3) Average Delay in the Inter-controller: The average propagation delay between switches in the inter-controller is calculated as:

$$T^{avg}=\frac{1}{2}\sum_{k=1}^{K}\frac{1}{\lvert V_k\rvert\lvert V/V_k\rvert}\sum_{j\in V/V_k}\sum_{i\in V_k}d(i,j) \tag{10}$$

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:14:41 UTC from IEEE Xplore. Restrictions apply.

where $d(i,j)$ is the propagation delay between switch i and switch j. And switch i and switch j belong to different controller domains.

4) Maximum Delay in the Inter-controller: The maximum propagation delay between switches in the inter-controller is calculated as:

$$T^{max} = \max_{k\in\{1,2,\ldots,K\}, i\in V_k, j\in V/V_k} d(i,j) \tag{11}$$

where switch i and switch j belong to different controller domains.

B. The Evaluation of Communication Consumption

In the same way, we can draw the equation for communication consumption as follows.

1) Average Communication Consumption Between Switches in the kth Intra-controller:

$$C_k^{avg} = \frac{1}{|V_k|\cdot(|V_k|-1)/2}\sum_{j>i}^{|V_k|}\sum_{i=1}^{|V_k|-1} c(i,j) \tag{12}$$

where $c(i,j)$ is the communication consumption between switch i and switch j.

For the entire network, the average communication consumption between switches in the intra-controller :

$$C_K^{avg} = \frac{1}{\sum_{k=1}^{K}\frac{1}{|V_k|\cdot(|V_k|-1)/2}}\sum_{k=1}^{K}\sum_{j>i}^{|V_k|}\sum_{i=1}^{|V_k|-1} c(i,j) \tag{13}$$

2) Maximum Communication Consumption in the kth Intra-controller:

$$C_k^{max} = \max_{i\in V, j\in V_k/i} c(i,j) \tag{14}$$

For the entire network, the average communication consumption between switches in the intra-controller:

$$C_K^{max} = \max_{k\in\{1,2,\ldots,K\}, i\in V_k, j\in V_k/i} c(i,j) \tag{15}$$

3) Average Communication Consumption in the Inter-controller:

$$C^{avg} = \frac{1}{2}\sum_{k=1}^{K}\frac{1}{|V_k||V/V_k|}\sum_{j\in V/V_k}\sum_{i\in V_k} c(i,j) \tag{16}$$

4) Maximum Communication Consumption in the Inter-controller:

$$C^{max} = \max_{k\in\{1,2,\ldots,K\}, i\in V_k, j\in V/V_k} c(i,j) \tag{17}$$

C. Simulation Results

Fig. 4 to Fig. 7 show the performance comparison of the algorithms at different numbers of controllers. Fig. 4 and Fig. 5 show the delay and consumption for random clustering algorithm, k-mean algorithm, CFSFDP algorithm, DBCP algorithm and our proposed algorithm in the intra-controller at different numbers of controllers. Fig. 6 and Fig. 7 show the delay and consumption for our proposed algorithm and four compared algorithms in the inter-controller at different numbers of controllers. From these figures, we can see that as the number of controllers increases, the average delay, maximum delay, average consumption, and maximum consumption of the five algorithms become smaller. As the switches are divided into more clusters, the number of switches in each cluster is smaller. The switches are managed more finely by the controllers. The performance of the random clustering algorithm is the worst. Because the number and location of the controllers and the clustering of the switches are all randomly assigned.

As shown in Fig. 4 and Fig. 5, when the number of controllers is 1, the average delay and average consumption of the k-mean algorithm, CFSFDP algorithm, DBCP algorithm and our proposed algorithm are almost the same. When the number of controllers is $k > 3$, the CFSFDP algorithm achieves better performance than k-mean algorithm in terms of latency and consumption. The core idea of the CFSFDP algorithm lies in the description of the clustering center. The k-mean algorithm randomly selects the clustering centers as the initialization, and the selection of the initial clustering centers seriously affects the quality of the clustering results. With the increase in the number of controllers, the shortcomings of the k-mean algorithm are gradually emerged. The latency between switches is defined as the number of hops between switches in the DBCP algorithm and does not consider the consumption between nodes. So the performance of the DBCP algorithm is worse than our proposed algorithm. The proposed algorithm has better performance than the random clustering algorithm, k-mean algorithm, CFSFDP algorithm and DBCP algorithm, because it considers the propagation delay and communication consumption separately in the process of selecting the location of the controller and the switch node clustering. From the above figures, it can be concluded that the proposed algorithm is superior to other reference algorithms in terms of propagation delay and communication consumption.

V. Conclusion

In this paper, we propose a controller placement algorithm based on density clustering algorithm in SDN. We take the propagation delay and communication consumption as the optimized objectives of the controller deployment in the intra-controller and inter-controller. The propagation delay matrix and the communication consumption matrix are separately introduced into the distance matrix when controllers are selected and switches are clustered. Moreover, the inflection point is automatically identified by the change of the cosine value, thereby automatically determining the number and location of controllers. Simulation results show that our proposed algorithm can not only determine the number and location of controllers but also have better performance than the compared algorithms in terms of the propagation delay and the communication consumption. In the future work, we plan to consider the impact of cutoff distance d_c on network performance.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:14:41 UTC from IEEE Xplore. Restrictions apply.

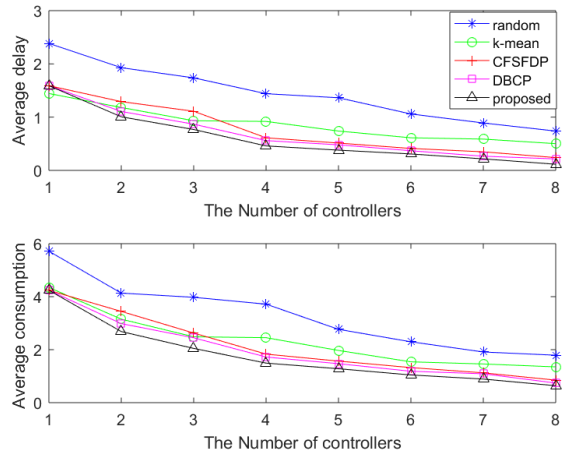

Fig. 4. Average delay and average consumption in the intra-controller at different numbers of controllers.

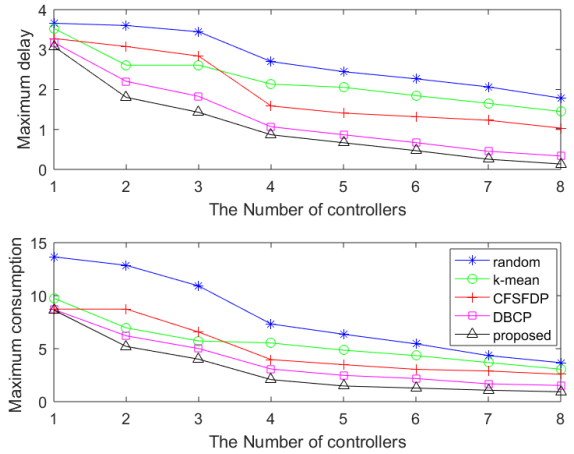

Fig. 5. Maximum delay and maximum consumption in the intra-controller at different numbers of controllers.

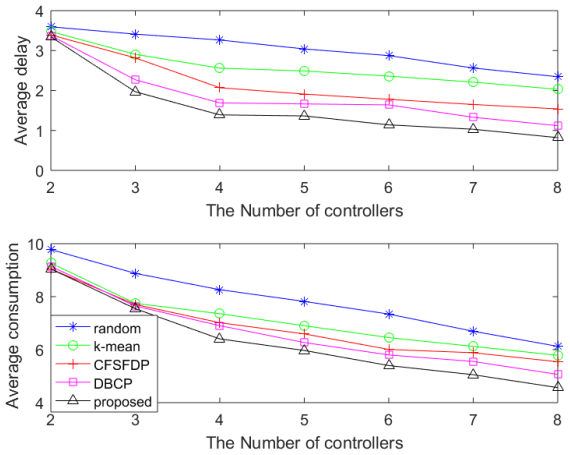

Fig. 6. Average delay and average consumption in the inter-controller at different numbers of controllers.

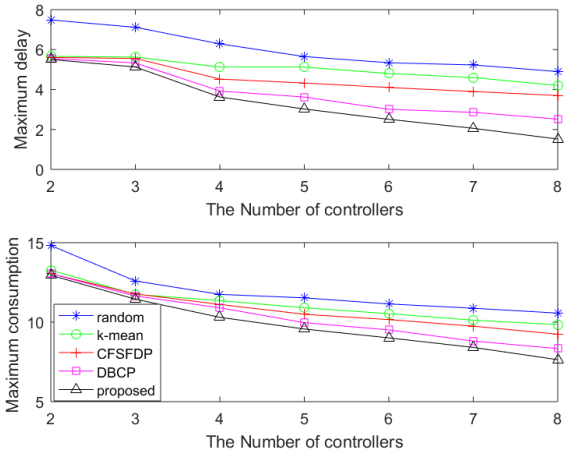

Fig. 7. Maximum delay and maximum consumption in the inter-controller at different numbers of controllers.

ACKNOWLEDGMENT

This work is supported by the 111 project(NO.B17007) and Director Funds of Beijing Key Laboratory of Network System Architecture and Convergence(NO.2017BKL-NSAC-ZJ-01).

REFERENCES

[1] D. Kreutz, F. M. V. Ramos, P. Esteves Verissimo, C. Esteve Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-Defined Networking: A Comprehensive Survey," Proceedings of the IEEE, vol. 103, no. 1, pp. 10-13, 2014.

[2] G. Wang, Y. Zhao, J. Huang, and W. Wang, "The Controller Placement Problem in Software Defined Networking: A Survey," IEEE Network, vol. 31, no. 5, pp. 21-27, 2017.

[3] A. Tootoonchian and Y. Ganjali, "HyperFlow: a distributed control plane for OpenFlow," in Internet Network Management Conference on Research on Enterprise NETWORKING, 2010, pp. 3-3.

[4] T. Koponen, M. Casado, N. Gude, J. Stribling, L. Poutievski, M. Zhu, et al., "Onix: a distributed control platform for large-scale production networks," in Usenix Conference on Operating Systems Design and Implementation, 2010, pp. 351-364.

[5] T. Y. Cheng, M. Wang, and X. Jia, "QoS-Guaranteed Controller Placement in SDN," in IEEE Global Communications Conference, 2015, pp. 1-6.

[6] Y. Hu, T. Luo, N. Beaulieu, and C. Deng, "The Energy-Aware Controller Placement Problem in Software Defined Networks," IEEE Communications Letters, vol. 21, no. 4, pp. 741-744, 2017.

[7] A. Ksentini, M. Bagaa, T. Taleb, and I. Balasingham, "On using bargaining game for Optimal Placement of SDN controllers," in IEEE International Conference on Communications, 2016, pp. 1-6.

[8] J. Liu and R. Xie, "Reliability-based controller placement algorithm in software defined networking," Computer Science & Information Systems, vol. 13, no. 2, pp. 547-560, 2016.

[9] J. Liao, H. Sun, J. Wang, Q. Qi, K. Li, and T. Li, "Density cluster based approach for controller placement problem in large-scale software defined networkings," Computer Networks, vol. 112, pp. 24-35, 2017.

[10] A. Rodriguez and A. Laio, "Clustering by fast search and find of density peaks," Science, vol. 344, no. 6191, pp. 1492-1496, 2014.

[11] K. R. Zalik, "An efficient k'-means clustering algorithm," Pattern Recognition Letters, vol. 29, no. 9, pp. 1385-1391, 2008.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:14:41 UTC from IEEE Xplore. Restrictions apply.